

Jordan's Ticket Triage Assistant

Project Build Log - Steps Executed

Author: Jeffrey De La Cruz

Date: 22 July 2026

Repository: Customer Support Specialist Week 3 and 4

An automated triage layer that scores, sorts, and explains a support queue before an agent opens it. This document records every step executed to build the project, in order, with the exact commands and their outcomes.

1. Overview

The project was built from a written specification (PROJECT_PLAN.md and CLAUDE.md) in six stages, each ending in one Conventional-Commits git commit so the build reads top-to-bottom in the git log. The pipeline: a rule-based priority scorer with a Critical-class safety net, a Claude rationale layer, a batch classification job, and a Streamlit review UI.

Key decisions made before building

- Data: the real Kaggle multilingual support-ticket dataset was provided and used (20,000 rows -> 11,923 English tickets).
- Claude API: the rationale layer was written but NOT run (no API key); the pipeline runs score-only with a deterministic fallback.
- Location: built in-place and initialised as its own standalone git repo.
- Critical label: the dataset has only low/medium/high (no 'Critical'). Resolution: treat dataset 'high' as the top-severity bucket; the product still emits Critical via the safety net; ground truth uses dataset labels only.

Environment

- Python 3.9.6; a virtual environment (venv) was created.
- Dependencies installed: pandas, streamlit, anthropic SDK, python-dotenv, pytest.

2. Stage 1 - Scaffold and data preparation

Created the repo structure, virtual environment, dependency and ignore files, shared data types (schema.py), and the data-prep script. Relocated the ~60 MB of raw dataset CSVs into data/raw/ (git-ignored).

```
python3 -m venv venv
./venv/bin/pip install -r requirements.txt
git init
python src/data_prep.py
```

Outcome: data/processed/tickets.csv created with 11,923 English tickets and a mapped priority column (high 4,571 / medium 4,952 / low 2,400).

Commit: chore: scaffold project, filter English-only tickets, map priority labels

3. Stage 2 - Rule-based scorer and Critical-class safety net

Built the deterministic scorer (0-100 urgency score + category) and the safety net: any ticket containing a hard-priority signal (outage, breach, payment failed, locked out, and similar) is force-escalated to at least High regardless of score. Added a pytest suite, including one parametrised test per hard-trigger signal.

```
./venv/bin/python -m src.rule_scorer # smoke test on sample tickets
./venv/bin/python -m pytest -q      # 19 tests passing
```

Commit: feat: rule-based priority scorer with Critical-class safety net

4. Stage 3 - Claude rationale layer and batch job

Wrote llm_rationale.py: given the fixed rule-based score, it asks Claude (model claude-opus-4-8) for a short 'why' and a suggested queue, using structured JSON output. The LLM explains the score; it never re-decides it. A deterministic rule-based fallback is used when no API key is present, so there is no single point of failure. batch_classify.py runs the scorer over the whole dataset into

classified_tickets.json.

```
./venv/bin/python -m src.batch_classify          # score-only, full dataset
./venv/bin/python -m src.batch_classify --limit 800 \
  --output data/processed/classified_tickets.sample.json
```

Outcome: 11,923 tickets classified. The full 12 MB output is git-ignored; an 800-ticket sample is committed so the demo runs on clone.

Commit: feat: add Claude API rationale generation and batch classification pipeline

5. Stage 4 - Streamlit triage UI

Built app.py: a priority-sorted queue (urgency-score bar, simulated SLA countdown, safety-net flag), a ticket detail view with the rationale and suggested queue, and a confirm/adjust control that logs every agent override to results/override_log.csv - the agent always has final say. Added a Streamlit AppTest smoke test.

```
./venv/bin/python -m pytest -q                # 21 tests passing
./venv/bin/streamlit run app.py               # served HTTP 200
```

Commit: feat: build triage queue UI with confirm/adjust control

6. Stage 5 - Evaluation and tuning

Wrote evaluate.py to score the full dataset and compare against the dataset's ground-truth labels. A term-frequency analysis by priority showed top-severity tickets cluster on infrastructure/outage/security vocabulary, so the keyword sets and weights were tuned recall-first (prioritising the top-severity class over overall accuracy).

```
./venv/bin/python -m src.evaluate    # writes results/evaluation.md
```

Measured results (11,923 tickets)

- Top-severity (Critical-class) recall: 24.7% -> 40.2% after tuning.
- Overall accuracy: 29.2% -> 40.6% after tuning.
- Per-class recall: high 40.2%, medium 49.6%, low 22.7%.
- Provable guarantee: 100% of hard-trigger tickets escalate to at least High (verified by tests).
- A 25-ticket spot-check sample was written for human review; no agreement number was fabricated.

Commit: test: evaluate scorer accuracy, confusion matrix, and Critical-class recall

7. Stage 6 - Documentation

Wrote README.md (technical: problem, architecture, real results, setup, limitations) and CASE_STUDY.md (narrative: the 5-Whys root cause, what was built, measured impact) - both using the real computed numbers, with no placeholder metrics.

Commit: docs: add README with architecture, results, and demo instructions

8. Git commit history (reads as the build story)

```
docs:  add README with architecture, results, and demo instructions
test:  evaluate scorer accuracy, confusion matrix, Critical-class recall
feat:  build triage queue UI with confirm/adjust control
feat:  add Claude API rationale generation and batch classification
feat:  rule-based priority scorer with Critical-class safety net
chore: scaffold project, filter English-only tickets, map priority labels
```

Note: the final v1.0 tag is intentionally deferred until the live Claude rationale batch is run with an API key.

Final verification

- Full test suite: 24 tests passing.
- Pipeline runs end-to-end on the real data (data prep -> batch -> evaluation -> UI).
- Working tree clean; secrets, venv, raw data, and the full JSON are git-ignored.

Appendix A - How to open and run the project

Open the folder

- Finder: Downloads > jeffrey-portfolio > 'Customer Support Specialist Week 3 and 4'.
- VS Code: File > Open Folder..., then select that folder (or run: code "<path to the folder>").

Run it (macOS / Linux)

Open Terminal, change into the project folder, then:

```
cd "<path to the folder>"
source venv/bin/activate      # venv already exists in this copy
streamlit run app.py         # opens the triage UI in your browser
```

Other useful commands:

```
python src/data_prep.py      # rebuild data/processed/tickets.csv
python -m src.batch_classify  # regenerate classified_tickets.json
python -m src.evaluate        # reproduce the evaluation numbers
pytest -q                    # run the test suite
```

If the venv does not work (for example on a different computer), recreate it:

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

On Windows, activate the venv with: venv\Scripts\activate

Appendix B - How to move the project to a flash drive

A flash-drive copy should be self-contained but should NOT include the venv folder (it is large and tied to this computer - recreate it on the other machine). Do include data/raw so the data step can run anywhere.

Easiest way (Finder)

- Plug in the flash drive; it appears in Finder under 'Locations'.
- Optionally delete the 'venv' folder inside the project first to save space, or just copy the whole folder and ignore venv.
- Drag the 'Customer Support Specialist Week 3 and 4' folder onto the flash drive to copy it.
- When the copy finishes, click the eject arrow next to the drive before unplugging.

Cleaner way (Terminal) - copies everything except venv and caches

```
rsync -av --exclude venv --exclude __pycache__ \
  --exclude '.pytest_cache' \
  "<path to the folder>" \
  /Volumes/YOUR_DRIVE_NAME/
```

Replace YOUR_DRIVE_NAME with the flash drive's name (see it under /Volumes/ or in Finder). Then eject the drive safely.

On the other computer

- Copy the project folder from the flash drive onto the computer's disk (running from a flash drive is slower).
- Open Terminal (or PowerShell on Windows) in the folder and recreate the environment: python3 -m

venv venv, activate it, pip install -r requirements.txt.

- Then run: streamlit run app.py

Tip: flash drives are usually formatted exFAT, which works fine here. Avoid very old FAT32 drives only if a single file exceeds 4 GB (none here do).